Parallelizzazione dell'algoritmo K-Means

OpenMP e CUDA: analisi di speedup, efficienza e scalabilità

Simone Brunelli - Matricola 0001021672

Anno accademico 2025/2026

1. Introduzione

Il progetto implementa e analizza tre versioni parallele dell'algoritmo **K-Means di Lloyd** per il clustering di dati multidimensionali: una versione **seriale** di riferimento, una versione **OpenMP** (parallelismo a memoria condivisa) e una versione **CUDA** (parallelismo massivo su GPU). I test sono stati eseguiti sul server **isi-raptor03.csr.unibo.it** (Intel Xeon E5-2603, 12 core a 1.70 GHz, 64 GB RAM, GPU NVIDIA GTX 1070).

Viene inoltre presentata, a scopo comparativo, una versione **MPI.** Tale implementazione non è oggetto di consegna.

Il carico computazionale dominante è il calcolo delle distanze euclidee nella fase di *classificazione*: per ogni iterazione ogni punto deve essere confrontato con tutti i K centroidi, per un costo O(N × K × D) per iterazione. La fase di *aggiornamento* dei centroidi introduce dipendenze di sincronizzazione che rappresentano il principale collo di bottiglia per la scalabilità.

Riproducibilità: tutte le versioni usano lo stesso seed (srand(123)), la stessa tolleranza di convergenza (TOL = 10^{-5}) e lo stesso limite di iterazioni (MAXITER = 100), garantendo che ogni implementazione esegua lo stesso numero di iterazioni sullo stesso dataset.

2. Versione OpenMP

2.1 Strategia di parallelizzazione

La parallelizzazione con OpenMP agisce sulle due funzioni computazionalmente costose:

classify(): il ciclo sui punti è completamente parallelo, ogni thread calcola le distanze per un sottoinsieme di punti. L'aggiornamento di *counts[]* è protetto con una **reduction su array**, evitando sezioni critiche nel loop principale.

update_centroids(): ogni thread alloca array locali (*local_nc* e *local_cnt*) per accumulare i contributi in modo indipendente. Il merge nella struttura condivisa avviene in una **#pragma omp critical**, riducendo la contesa rispetto all'uso di operazioni atomiche per ogni punto.

2.2 Prestazioni: strong scaling

I test di strong scaling utilizzano il dataset **test-N150000-D20.in** (N=150.000, D=20, K=5). I tempi sono il minimo di 3 esecuzioni.

p (thread)	T_ser (ms)	T_OMP (ms)	S_OMP	E_OMP	Note
1	5162	5294	0.98	0.98	-
2	5162	2667	1.94	0.97	-
4	5162	1339	3.86	0.96	ottimale
8	5162	1663	3.10	0.39	degrado NUMA
12	5162	1144	4.51	0.38	degrado

Tabella 1 - Strong scaling OpenMP su test-N150000-D20.in (K=5).

Come si osserva in Tabella 1 e Figura 1, OpenMP scala efficacemente fino a **4 thread** (efficienza 96%). Oltre questa soglia le prestazioni degradano, mentre a 8 thread l'efficienza crolla a **39%**. La

causa è duplice: **contesa sulle sezioni critiche** durante il merge dei centroidi, che scala male con il numero di thread, effetti **NUMA,** il processore Xeon E5-2603 accede alla memoria con latenze diverse a seconda del socket, e con 8+ thread si attraversa il bus inter-socket.

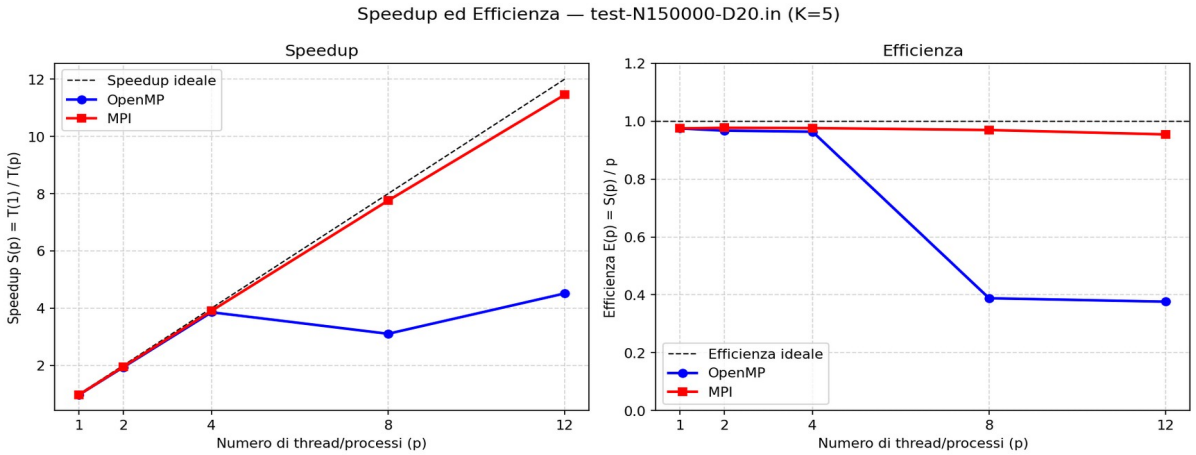


Figura 1 - Speedup (sinistra) ed efficienza (destra) di OpenMP e MPI su test-N150000-D20.in (K=5).

Per confronto, MPI su 12 processi raggiunge speedup **11.45×** con efficienza **95.5%**: la memoria distribuita elimina la contesa e ciascun processo comunica solo i centroidi aggregati tramite *MPI_Allreduce*, con overhead trascurabile rispetto al calcolo.

3. Versione CUDA

3.1 Strategia di parallelizzazione

L'implementazione CUDA decompone l'algoritmo in quattro kernel distinti:

kernel_classify: ogni thread GPU gestisce un singolo punto, calcolando la distanza da tutti i K centroidi. I centroidi vengono caricati cooperativamente in **shared memory** all'inizio di ogni blocco (tutti i thread del blocco si dividono la copia dalla global memory), eliminando qualsiasi vincolo fisso su K e D. Se K×D supera i 48 KB disponibili, si usa la global memory sfruttando la cache L2.

kernel_accumulate: riduzione per blocco in shared memory. Ogni blocco accumula i propri punti localmente (**atomicAdd** su shared memory, senza contesa inter-blocco), poi esegue un unico **atomicAdd** per blocco sulla global memory. Le operazioni atomiche globali scendono da N×D a (N/256)×K×D.

kernel_divide e **kernel_convergence**: aggiornamento dei centroidi e calcolo dello spostamento massimo (riduzione finale su host).

3.2 Prestazioni e weak scaling

La GTX 1070 (2048 CUDA core, 8 GB GDDR5) mostra prestazioni dipendenti dalla dimensione del dataset. Su **test-N150000-D20.in** (K=5) il tempo è **304 ms** contro 5169 ms del seriale (speedup **17×**). Su **letter-recognition.in** lo speedup sale a **69×** (1224 ms seriale vs 17.7 ms CUDA). Su **test-N50000-D80.in** invece CUDA è solo **1.6×** più veloce (506 ms vs 792 ms): con D=80 e N=50.000 la GPU non raggiunge piena occupancy degli Streaming Multiprocessor, e il costo del caricamento cooperativo dei centroidi in shared memory non viene ammortizzato a sufficienza.

Il test di weak scaling CUDA varia N da 50.000 a 3.200.000 punti (K=10, D=2):

Moltiplicatore	N punti	Blocchi GPU	Tempo (ms)	Ratio vs N=100k
1×	50.000	196	53.9	2.23×
2×	100.000	391	24.1	1.00× (rif.)
4×	200.000	782	49.5	2.05×

8×	400.000	1.563	76.7	3.18×
16×	800.000	3.125	148.2	6.15×
32×	1.600.000	6.250	254.8	10.6×
64×	3.200.000	12.500	473.6	19.6×

Tabella 2 - Weak scaling CUDA: N cresce da 50k a 3.2M punti (K=10, D=2).

Il dato anomalo a N=50.000 (53.9 ms, più lento di N=100.000) è dovuto al **GPU warm-up**: la prima esecuzione paga il costo di inizializzazione del contesto CUDA e del caricamento dei kernel JIT. Escludendo questo punto, la crescita è **nettamente sublineare**, N cresce di 32× da 100k a 3.2M, ma il tempo cresce solo di 19.6×. Ciò dimostra che la GPU aumenta l'occupancy degli Streaming Multiprocessor al crescere di N, migliorando l'efficienza complessiva.

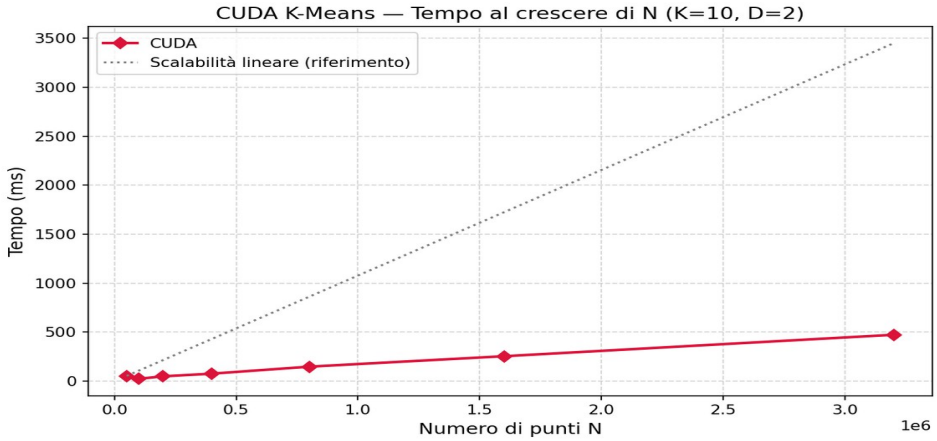


Figura 2 - Weak scaling CUDA: tempo al crescere di N. La curva reale (verde) è nettamente al di sotto della scalabilità lineare (grigio).

4. Weak Scaling OpenMP e MPI

Il test di weak scaling CPU utilizza N = 100.000 × p punti per p unità di esecuzione (K=10, D=2). L'efficienza ideale è 1.0, il tempo rimane costante al crescere del carico.

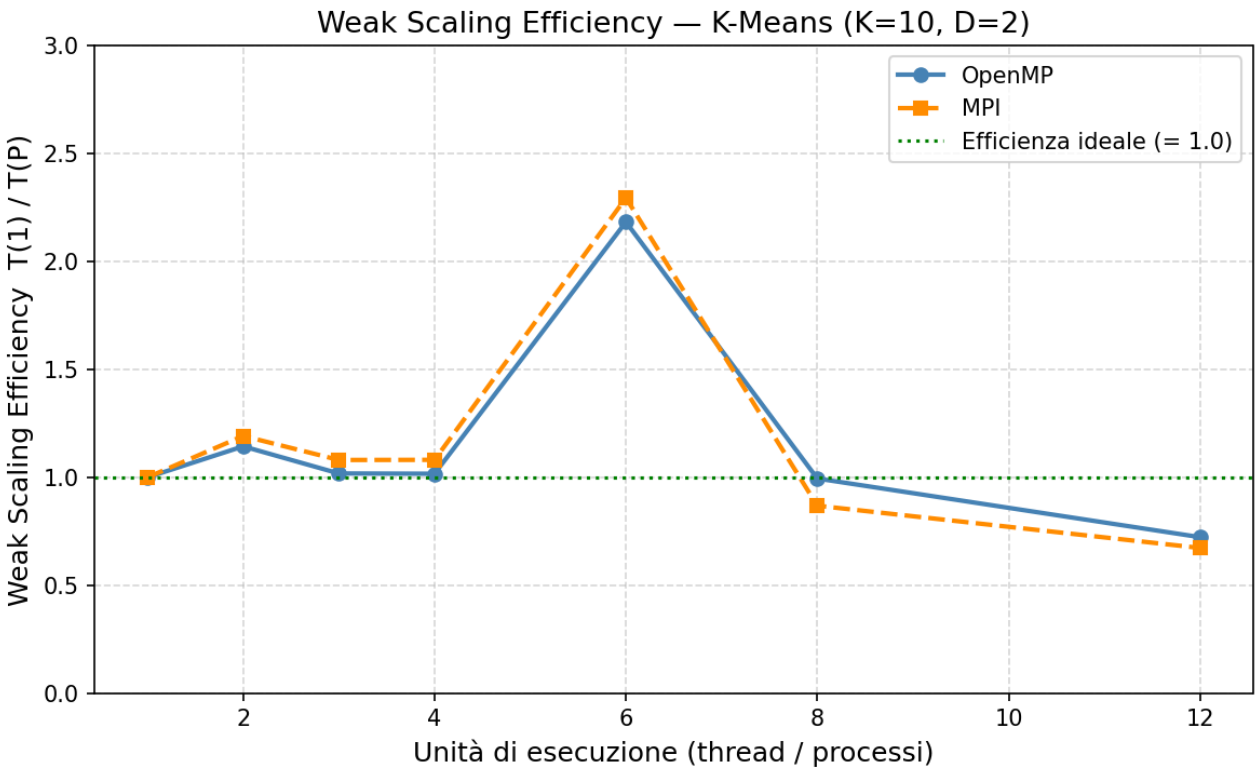


Figura 3 - Weak scaling efficiency di OpenMP e MPI (K=10, D=2, N_base=100k).

Entrambe le implementazioni mantengono efficienza vicina a 1.0 fino a 4 unità. Il picco superiore a 1.0 per 2-4 unità (efficienza fino a ~1.2) indica un **effetto cache** favorevole: con dataset più grandi la distribuzione dei punti è più uniforme, riducendo il numero di iterazioni necessarie alla convergenza.

Il picco anomalo a **6 unità** (efficienza ~2.2) è spiegabile con un effetto di **"cache-resident"**: a 6 core/processi la suddivisione del dataset corrisponde a un'occupazione ottimale della cache L3 disponibile (probabilmente 6 core per socket NUMA), riducendo drasticamente gli accessi alla RAM. Il calo a 8-12 unità riflette l'overhead inter-socket e la saturazione della banda di memoria.

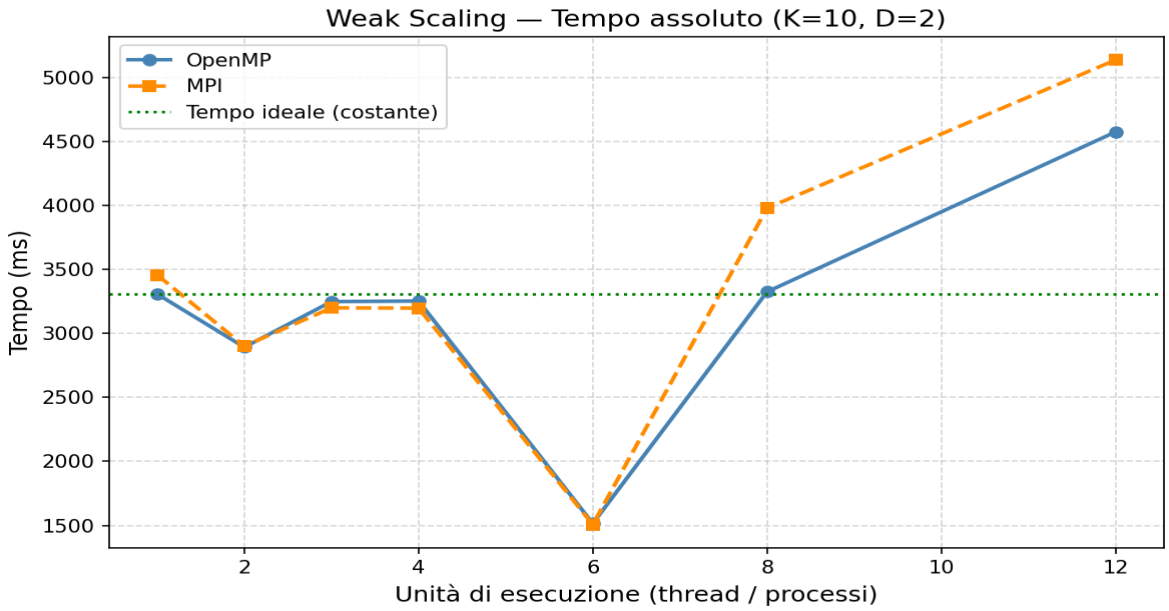


Figura 4 - Tempi assoluti weak scaling: il minimo a 6 unità evidenzia il "sweet spot" della cache L3.

5. Conclusioni

OpenMP è efficace fino a 4 thread (efficienza 96%) ma decade oltre a causa di effetti NUMA e overhead di sincronizzazione. È la soluzione più semplice da implementare e sufficiente per parallelismo moderato.

MPI scala quasi linearmente fino a 12 processi (speedup 11.45×, efficienza 95.5%). La memoria distribuita elimina la contesa e rende MPI la scelta migliore su CPU per dataset grandi.

CUDA offre lo speedup più elevato su dataset grandi (fino a 69× su letter-recognition.in). Lo speedup dipende fortemente da N e D: con N piccolo o D grande la GPU non raggiunge piena occupancy e il vantaggio si riduce (1.6× su test-N50000-D80.in). L'uso della shared memory per i centroidi e la riduzione per blocco nell'accumulo eliminano i vincoli su K e D.

La scelta del paradigma dipende dal contesto, per carichi moderati su CPU OpenMP è sufficiente, per la massima scalabilità su cluster MPI è preferibile, per dataset massivi su singola macchina con GPU, CUDA è nettamente superiore.